Chapter 1

Interpolation, Curve Fitting, and Regression

So far, when we have wanted a function that describes a relationship between two variables, we've written one ourselves — a polynomial we wrote out by hand, or a differential equation we integrated. Often, though, the relationship is not given to us as a formula but as a *table of measured values*. A thermometer reading at 1 PM, 2 PM, and 3 PM. A flight test that records airspeed every quarter second. A battery's terminal voltage sampled once per hour as it discharges.

In each case the data describe a function only at the discrete points that were measured, and we are usually interested in something more. Two questions come up over and over:

- Given the measured points, what value would the function have *between* the samples? This is the question *interpolation* answers.
- Given the measured points, is there a simple formula — a polynomial, an exponential, a sum of sines — that approximately describes the trend? This is the question *curve fitting* (and its statistical cousin *regression*) answers.

The two questions sound similar, and they overlap, but they reflect different goals. Interpolation tries to honor every data point exactly. Curve fitting accepts that the data may be noisy and looks for a model that summarizes the data even if the model misses individual points.

1.1 Interpolation

Imagine you logged a sensor reading once per second:

```
>> x = [1, 2, 3, 4, 5];
>> y = [2, 25, 20, 32, 40];
```

and you want to know, after the fact, what the sensor would have read at $x = 1.5$. The sample at $x = 1$ was 2 and the sample at $x = 2$ was 25, so a reasonable guess for $x = 1.5$ is

somewhere between 2 and 25.

The simplest assumption is that the underlying signal varies linearly between samples. In that case the interpolated value $\hat{y}$ at a query point x that lies between samples x_i and x_{i+1} is

$$\hat{y} = y_i + \frac{y_{i+1} - y_i}{x_{i+1} - x_i}(x - x_i).$$

This formula is just the equation of the line through the two adjacent samples, evaluated at x. MATLAB has a built-in function, `interp1`, that does this for you:

```
>> interp1(x, y, 1.5)
ans = 13.5
```

The first two arguments are the sample locations and the sample values; the third is the query point. You can also pass an entire vector of query points, in which case `interp1` returns a vector of the same length:

```
>> xq = linspace(1, 5, 25);
>> yq = interp1(x, y, xq);
```

Plotting the original samples and the interpolated curve gives a piecewise-linear connection of the dots — straight line segments between every pair of adjacent samples.

1.1.1 Cubic Spline Interpolation

Linear interpolation is fast and easy to understand, but the resulting curve has corners at every sample point — the slope changes abruptly. For data that we expect to vary smoothly, we'd rather have an interpolant that is smooth too.

Cubic spline interpolation fits a separate cubic polynomial between every pair of adjacent samples, with the constraint that the cubics meet at the samples and have continuous first and second derivatives. In each interval the interpolant looks like

$$S_i(x) = a_i x^3 + b_i x^2 + c_i x + d_i,$$

and the algorithm chooses the coefficients so that the pieces join smoothly. The result passes through every sample point and looks like a single, smooth curve.

MATLAB's `spline` function does this with the same calling convention as `interp1`:

```
>> yq = spline(x, y, xq);
```

Equivalently, you can call `interp1(x, y, xq, 'spline')` — `interp1` accepts a fourth argument that selects the interpolation method (`'linear'`, `'spline'`, `'pchip'`, and a few others). See the `interp1` documentation for the full list.

For the same five samples used above, linear interpolation produces a connect-the-dots polyline, while a cubic spline produces a smooth curve that nevertheless still passes through

each sample. The two interpolants agree at the samples, by construction; they disagree everywhere in between.

1.1.2 When to Use Which?

Linear interpolation is a safe default when you have many samples and you don't expect the underlying signal to be smoother than the sampling rate. Cubic splines look better and have continuous derivatives, but they can also *overshoot* — excursions above or below all of the data — when the data are not smooth. If you have noisy samples and you fit a spline through every one of them, the interpolant will faithfully reproduce the noise as wiggles between samples. In that case, what you actually want is not interpolation but curve fitting.

1.2 Polynomial Curve Fitting

Suppose you measured the terminal voltage of a 12 V battery once per hour as it discharged under a constant load:

```
>> tt = [0 1 2 3 4 5 6 7 8 9];                    % hours
>> vv = [13.55 8.50 6.80 4.31 3.35 3.86 ...
         2.94 1.14 1.79 0.65];                    % volts
```

The data are clearly noisy — there's a stretch where the voltage briefly increases between hours 4 and 5 — but the overall trend is downward. An interpolant would honor every measured point, including the noise; instead, we'd like to find a smooth curve that summarizes the trend.

The simplest assumption is that the trend is a straight line: $v = m\,t + b$. This is a polynomial of degree 1, and MATLAB's `polyfit` function will find the slope m and intercept b that minimize the sum of the squared vertical distances from the data to the line:

```
>> n = 1;
>> p = polyfit(tt, vv, n)
p =
   -1.18    10.01
```

`polyfit` returns the polynomial coefficients in *descending* order of power, so we read `p` as $v(t) = -1.18\,t + 10.01$.

To plot the fitted line, we use `polyval`, which evaluates a polynomial at one or more query points:

```
>> t_fit = linspace(min(tt), max(tt), 100);
>> v_fit = polyval(p, t_fit);
>> plot(tt, vv, 'ko', t_fit, v_fit, 'r--')
```

The `ko` markers show the data; the dashed red line is the first-order polynomial (straight-line) fit. The line clearly captures the overall downward trend, even though it doesn't pass through any individual data point.

1.2.1 How Good is the Fit?

`polyfit` can return additional output that summarizes how well the fit describes the data. We're going to focus on one number, the *coefficient of determination* R^2:

```
>> [p, S] = polyfit(tt, vv, 1);
>> fprintf('R^2 = %.2f\n', S.rsquared);
R^2 = 0.82
```

R^2 ranges from 0 to 1. A value of 1 means the model passes through every data point. A value of 0 means the model does no better than just using the mean of the data — i.e., a horizontal line through $\bar{v}$. In between, R^2 tells you what fraction of the variability in the data is explained by the model. An R^2 of 0.82 says the first-order polynomial captures 82% of the variance; the remaining 18% is the part of the data the line doesn't account for. As a rule of thumb, engineering applications often look for $R^2 > 0.9$ before considering a fit "good enough," but the right threshold depends on the application.

1.2.2 Higher-Order Polynomials

A line is just the simplest case. We can fit a polynomial of any degree:

```
>> p2 = polyfit(tt, vv, 2);   % parabola
>> p3 = polyfit(tt, vv, 3);   % cubic
```

Higher-order polynomials have more flexibility, so they can fit the data more closely — at the cost of being more complicated and, often, less meaningful. As you raise the order, R^2 goes up, but at some point you start fitting the noise rather than the underlying trend. This is called *overfitting*.

The classic warning sign of overfitting is a model that performs beautifully on the training data and terribly on new data — like a student who memorized the practice problems rather than understanding the material. An overfit polynomial will wiggle aggressively to pass close to every data point, but the wiggles don't generalize to new measurements.

A good piece of advice for picking the polynomial order:

> "Everything should be made as simple as possible, but not simpler." —Albert Einstein (paraphrased)

Choose the lowest order that captures the structure you actually believe is there. If a line is enough, don't fit a parabola.

1.3 Nonlinear Regression

`polyfit` is a special case of a more general idea: *regression*, the statistical task of choosing model parameters to minimize the discrepancy between a model and a set of measurements. Polynomial fits are **linear with respect to (wrt) their parameters** — doubling a coefficient doubles the model's contribution from that term — which makes them tractable using a single matrix equation. But many physical models are not linear wrt their parameters.

The battery discharge example again: from physical reasoning we expect the voltage to decay exponentially toward a cutoff voltage, not linearly. A reasonable model is

$$V(t) = (V_0 - c)\, e^{-kt} + c,$$

which has three parameters: the initial voltage V_0, the decay rate k, and the cutoff voltage c. This model is *nonlinear* in k because k appears inside the exponential. We can't fit it with `polyfit`.

MATLAB provides `nlinfit` for this case. The interface looks like

```
>> params = nlinfit(x, y, @modelfun, params0)
```

where `modelfun` is a function that takes a parameter vector and a vector of x values and returns a vector of predicted y values, and `params0` is an initial guess for the parameters. The signature of `modelfun` is fixed:

```
function yhat = modelfun(beta, x)
```

— `beta` is the parameter vector and `x` is the independent-variable vector. For our battery model:

```
function v = batt_model(params, t)
    Vo = params(1);
    k  = params(2);
    c  = params(3);
    v  = (Vo - c) .* exp(-k .* t) + c;
end
```

Now we can ask `nlinfit` to find the best parameters:

```
>> params0 = [13.5, 0.25, 0.5];          % initial guess
>> params  = nlinfit(tt, vv, @batt_model, params0)
params =
    13.20     0.40     1.05
```

We interpret the result as $V_0 = 13.2\,\mathrm{V}$, $k = 0.40\,1/\mathrm{hr}$, and $c = 1.05\,\mathrm{V}$. The time constant of the discharge is $1/k \approx 2.5$ hours.

Why an initial guess? Unlike the linear-wrt-parameters case, nonlinear regression is an iterative algorithm — it starts from a guess and refines it. If the initial guess is far from the true parameters, the algorithm can get stuck at a poor local minimum, or fail to converge. A reasonable initial guess made from looking at the data goes a long way.

1.3.1 Goodness of Fit for Nonlinear Models

`nlinfit` can also report a measure of fit quality. Calling it with additional output arguments returns the residuals (the differences between data and model) and a covariance estimate:

```
>> [params, resid, ~, ~, MSE] = nlinfit(tt, vv, @batt_model, params0);
>> fprintf('MSE = %.3f V^2\n', MSE);
```

The *mean squared error* (MSE) is the average of the squared residuals. Smaller is better; the units are the square of the units of the dependent variable. An MSE of $0.5\,\mathrm{V}^2$ means the typical residual has magnitude $\sqrt{0.5} \approx 0.7\,\mathrm{V}$ — the model misses each data point by about three quarters of a volt on average.

You can also compute R^2 for a nonlinear fit, just as for a polynomial fit, by hand:

```
v_predicted = batt_model(params, tt);
SSE = sum((vv - v_predicted).^2);
SST = sum((vv - mean(vv)).^2);
R2 = 1 - SSE/SST;
```

The interpretation is the same as for `polyfit`: the fraction of variance in the data explained by the model.

1.4 Curve Fitting vs. Regression

The two terms are sometimes used interchangeably, but they emphasize different things. *Curve fitting* is a general numerical technique — find a curve, of whatever form is convenient (polynomial, spline, exponential, sinusoidal sum) that approximates the data. It is often justified by an engineering or physical argument: if the underlying process is exponential, fit an exponential. *Regression* is a statistical procedure that not only finds the best parameters but also lets you ask probabilistic questions: how confident are we in the parameter estimates? how likely is it that an effect we see in the data is real rather than noise? Regression assumes a probabilistic model of the noise, and it lets you compute things like confidence intervals and p-values.

For everyday engineering work, the distinction often doesn't matter much — you find the parameters that minimize the squared error and you call it a day. But if you ever need to defend a fit ("how do we know this is real and not noise?") the answer comes from the regression machinery, not from R^2 alone.

1.5 Chapter Review

This chapter introduced two related families of techniques for working with measured data. *Interpolation* produces a function that passes exactly through your samples, useful when you need to estimate values between samples and you trust the data. MATLAB provides `interp1` for piecewise-linear (and other) interpolation, and `spline` for smooth cubic-spline interpolation.

Curve fitting produces a function that approximates your data using a model with a small number of parameters. When the model is a polynomial, `polyfit` finds the best-fit coefficients in the least-squares sense; `polyval` evaluates the polynomial at query points. When

the model is nonlinear wrt its parameters, `nlinfit` finds the parameters by iterative optimization, starting from an initial guess.

The coefficient of determination R^2 summarizes how well a model captures the variability in the data. An R^2 near 1 is good; an R^2 near 0 is poor. But high R^2 alone does not guarantee a useful model: an over-flexible model can chase noise rather than describe trend, a problem called *overfitting*. The right model is the simplest one that captures the structure you believe is in the data.

1.6 Exercises

Exercise 1.1. The `interp1` function takes an optional fourth argument that selects the interpolation method. Reread the data set used in Section 1:

```
x = [1, 2, 3, 4, 5];
y = [2, 25, 20, 32, 40];
xq = linspace(1, 5, 100);
```

For each of the methods `'linear'`, `'spline'`, `'pchip'`, and `'nearest'`, plot the interpolated curve over the original samples on the same set of axes. Which methods produce a smooth curve? Which methods overshoot?

Exercise 1.2. Using the battery data from Section 2:

```
tt = [0 1 2 3 4 5 6 7 8 9];
vv = [13.55 8.50 6.80 4.31 3.35 3.86 2.94 1.14 1.79 0.65];
```

fit a polynomial of degree 1, 2, 3, and 7. For each fit, report the R^2 value and plot the fitted polynomial against the data. Comment on the trade-off between fit quality and model complexity. At what order does the fit start to look "wiggly"?

Exercise 1.3. The nonlinear regression in Section 3 worked well with the initial guess `params0 = [13.5, 0.25, 0.5]`. Try the same fit with progressively worse initial guesses — e.g. `[10, 1, 0]`, `[1, 1, 1]`, `[100, 0.01, 0]`. At what point does `nlinfit` fail to find the right parameters, or fail to converge at all? This is the price of fitting models that are nonlinear wrt their parameters.